Table of Contents

..... 1
---------- INPUT PARAMETERS ---------- 1
---------- 1. ARRAY DATA MODEL ---------- 2
---------- 2. FFT-BASED SPATIAL SPECTRUM ---------- 2
---------- 3. BEAMFORMER WEIGHTS ---------- 2
---------- 4. STEERING CASES ---------- 3
---------- 5. PERFORMANCE METRICS ---------- 3
---------- 6. PLOTS ---------- 4
---------- 7. OUTPUT PARAMETER TABLE ---------- 9
---------- LOCAL FUNCTIONS ---------- 11


```
clc;
clear;
close all;
rng(1);
```

---------- INPUT PARAMETERS ----------

```
fc = 28e9; % carrier frequency (Hz)
c = 3e8; % speed of light (m/s)
lambda = c/fc; % wavelength (m)

N = 8; % number of antenna elements
d = lambda/2; % element spacing

thD = 20; % desired direction (deg)
thI = [-30 45]; % interference directions (deg)

steerCases = [-30 0 20 45]; % steering angles to compare (deg)

K = 2000; % number of snapshots

SNRdB = 10; % desired-signal SNR per element
INRdB = 10; % each interferer's INR per element

noiseVar = 1; % noise power per element
Nfft = 2048; % zero-padded spatial FFT length

n = (0:N-1).';

a = @(t) exp(1j*n*2*pi*d/lambda*sind(t));

th = -90:0.05:90;

A = a(th);
```

---------- 1. ARRAY DATA MODEL ----------

```
Pd = noiseVar*10^(SNRdB/10);
Pi = noiseVar*10^(INRdB/10);

cg = @(M,L) (randn(M,L) + 1j*randn(M,L))/sqrt(2);

Xd = a(thD)*(sqrt(Pd)*cg(1,K));

Xi = a(thI)*(sqrt(Pi)*cg(2,K));

Xn = sqrt(noiseVar)*cg(N,K);

X = Xd + Xi + Xn;
```

---------- 2. FFT-BASED SPATIAL SPECTRUM ----------

```
S = fftshift(mean(abs(fft(X, Nfft, 1)).^2, 2));

f = (-Nfft/2:Nfft/2-1).'/Nfft;

thS = asind(f*lambda/d);

SdB = 10*log10(S/max(S));

R = (X*X')/K;

Rinv = R\eye(N);

Pcap = 1./real(sum(conj(A).*(Rinv*A), 1));

CdB = 10*log10(Pcap/max(Pcap));

pk = top_peaks(SdB, 3);

doaFFT = sort(thS(pk)).';
```

---------- 3. BEAMFORMER WEIGHTS ----------

```
aD = a(thD);

w_conv = conj(aD)/N;

% Hamming taper
```

```
taper = 0.54 - 0.46*cos(2*pi*n/(N-1));

w_hamm = taper.*conj(aD)/sum(taper);

% MVDR beamformer

Rl = R + 1e-3*trace(R)/N*eye(N);

w_mvdr = conj((Rl\aD)/(aD'*(Rl\aD)));

% Array factor

AF_conv = 20*log10(abs(w_conv.'*A) + 1e-6);

AF_hamm = 20*log10(abs(w_hamm.'*A) + 1e-6);

AF_mvdr = 20*log10(abs(w_mvdr.'*A) + 1e-6);

% FFT-based evaluation of conventional pattern

AF_fft = fftshift(ifft(w_conv, Nfft)*Nfft);

AFfftdB = 20*log10(abs(AF_fft) + 1e-6);
```

---------- 4. STEERING CASES ----------

```
AFsteer = zeros(numel(steerCases), numel(th));

peakAng = zeros(1, numel(steerCases));

for i = 1:numel(steerCases)

    w = conj(a(steerCases(i)))/N;

    AFsteer(i,:) = 20*log10(abs(w.'*A) + 1e-6);

    [~, ix] = max(AFsteer(i,:));

    peakAng(i) = th(ix);

end
```

---------- 5. PERFORMANCE METRICS ----------

```
mC = beam_metrics(th, AF_conv);

mH = beam_metrics(th, AF_hamm);
```

```
[~, ipk] = max(AFfftdB);

steerFFT = thS(ipk);

gainAt = @(w, t) 20*log10(abs(w.'*a(t)));

W = {w_conv, w_hamm, w_mvdr};

names = {'Conventional (FFT)', 'Hamming taper', 'MVDR (adaptive)'};

SINR_in = 10*log10(mean(abs(Xd(:)).^2)/ ...
    (mean(abs(Xi(:)).^2) + mean(abs(Xn(:)).^2)));

SINR_out = zeros(1,3);

for i = 1:3

    SINR_out(i) = sinr_db(W{i}, Xd, Xi, Xn);

end
```

---------- 6. PLOTS ----------

```
% Figure 1: Spatial spectrum

figure('Name','Spatial spectrum','Color','w', ...
    'Position',[40 40 900 480]);

plot(thS, SdB,'b','LineWidth',1.6);
hold on;

plot(th, CdB,'r--','LineWidth',1.1);

xline(thD,'g-','Desired 20^\circ', ...
    'LineWidth',1.4,'HandleVisibility','off');

xline(thI(1),'k:','Int. -30^\circ', ...
    'LineWidth',1.4,'HandleVisibility','off');

xline(thI(2),'k:','Int. 45^\circ', ...
    'LineWidth',1.4,'HandleVisibility','off');

plot(thS(pk), SdB(pk),'bv', ...
    'MarkerFaceColor','b','MarkerSize',8);

grid on;
xlim([-90 90]);
ylim([-40 3]);

xlabel('Arrival angle \theta (deg)');
ylabel('Normalised power (dB)');
```

```
title('FFT-based spatial spectrum of the 8-element array data');

legend('FFT spatial spectrum','Capon (reference)', ...
    'Detected FFT peaks','Location','south');

% Figure 2: Beam pattern steered to 20 deg

figure('Name','Beam pattern','Color','w', ...
    'Position',[80 80 1150 480]);

subplot(1,2,1);

plot(th, AF_conv,'b','LineWidth',1.6);
hold on;

plot(thS(1:25:end), AFfftdB(1:25:end),'ro','MarkerSize',4);

xline(thD,'g-','LineWidth',1.3,'HandleVisibility','off');

xline(thI,'k:','LineWidth',1.3,'HandleVisibility','off');

grid on;
xlim([-90 90]);
ylim([-50 5]);

xlabel('\theta (deg)');
ylabel('|AF| (dB)');

title(sprintf('Beam pattern steered to %d^\\circ (peak at %.1f^\\circ)', ...
    thD, steerFFT));

legend('Direct AF formula','FFT-based AF','Location','south');

subplot(1,2,2);

polarplot(deg2rad(th), max(AF_conv,-50),'b','LineWidth',1.5);
hold on;

pax = gca;

pax.ThetaZeroLocation = 'top';
pax.ThetaDir = 'clockwise';

thetalim([-90 90]);
rlim([-50 0]);

polarplot(deg2rad([thD thD]), [-50 0],'g','LineWidth',1.2);

polarplot(deg2rad([thI(1) thI(1)]), [-50 0],'k:','LineWidth',1.2);

polarplot(deg2rad([thI(2) thI(2)]), [-50 0],'k:','LineWidth',1.2);

title('Polar beam pattern (dB)');
```

```
% Figure 3: Beam steering comparison

figure('Name','Beam steering','Color','w', ...
    'Position',[120 120 900 480]);

plot(th, AFsteer.','LineWidth',1.5);
hold on;

grid on;
xlim([-90 90]);
ylim([-50 5]);

xlabel('\theta (deg)');
ylabel('|AF| (dB)');

title('Beam steering with the same array (phase-shift weights)');

legend(arrayfun(@(s) sprintf('Steer %d^\\circ', s), ...
    steerCases, 'UniformOutput', false), ...
    'Location','south');

% Figure 4: Interference suppression

figure('Name','Interference suppression','Color','w', ...
    'Position',[160 160 1150 480]);

subplot(1,2,1);

plot(th, AF_conv,'b','LineWidth',1.4);
hold on;

plot(th, AF_hamm,'m','LineWidth',1.4);

plot(th, AF_mvdr,'r','LineWidth',1.4);

xline(thD,'g-','Desired', ...
    'LineWidth',1.3,'HandleVisibility','off');

xline(thI(1),'k:','Int. -30^\circ', ...
    'LineWidth',1.3,'HandleVisibility','off');

xline(thI(2),'k:','Int. 45^\circ', ...
    'LineWidth',1.3,'HandleVisibility','off');

grid on;
xlim([-90 90]);
ylim([-70 5]);

xlabel('\theta (deg)');
ylabel('|AF| (dB)');

title('Beam patterns of the three beamformers');
```

```
legend(names,'Location','south');

subplot(1,2,2);

bar([SINR_in SINR_out]);

grid on;

set(gca,'XTickLabel', ...
    {'Input (per element)','Conventional','Hamming','MVDR'});

ylabel('SINR (dB)');

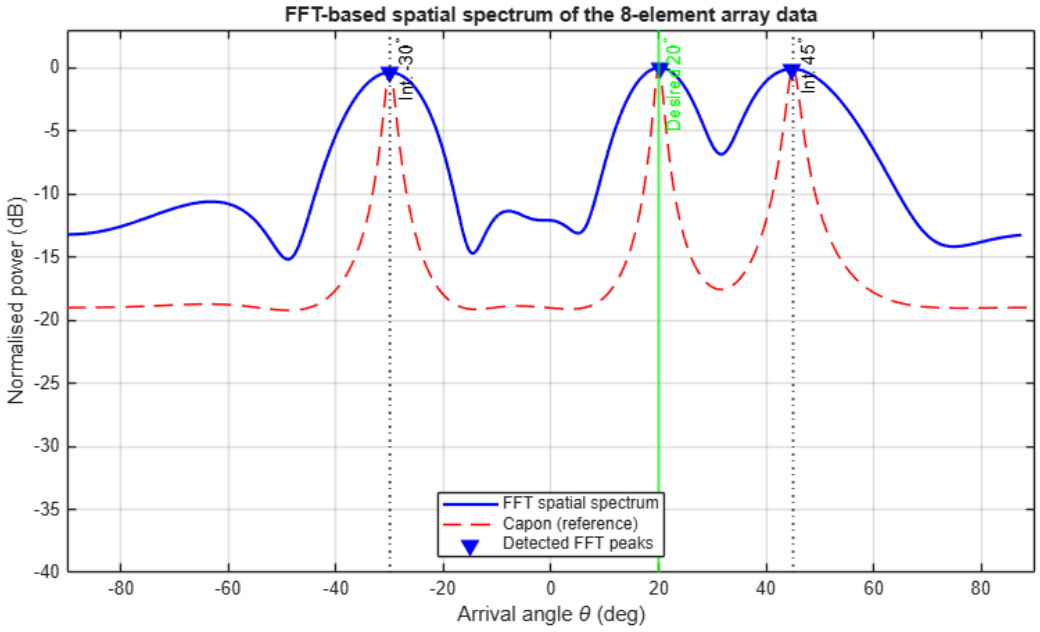
title('Output SINR');

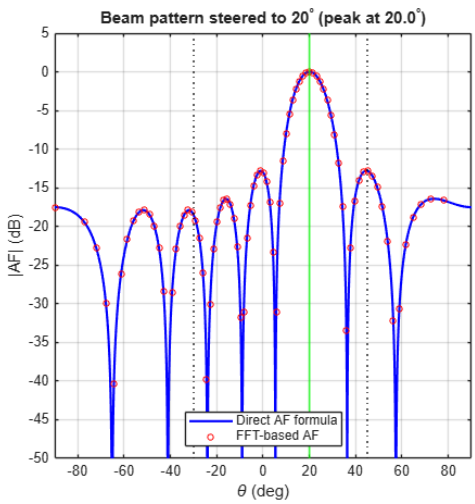
for i = 1:4

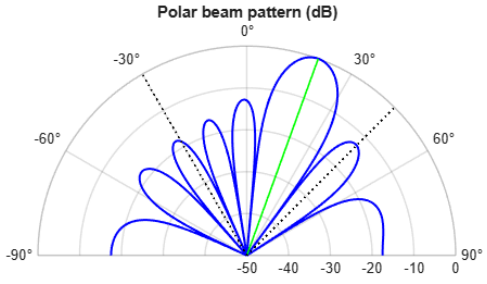
    v = [SINR_in SINR_out];

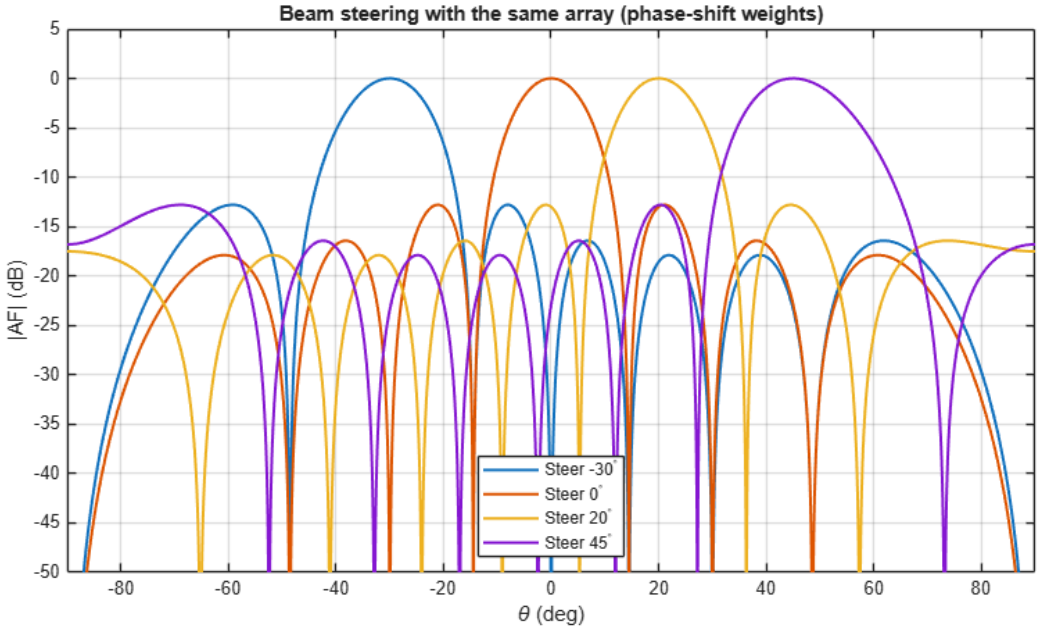
    text(i, v(i)+0.6, sprintf('%.1f', v(i)), ...
        'HorizontalAlignment','center');

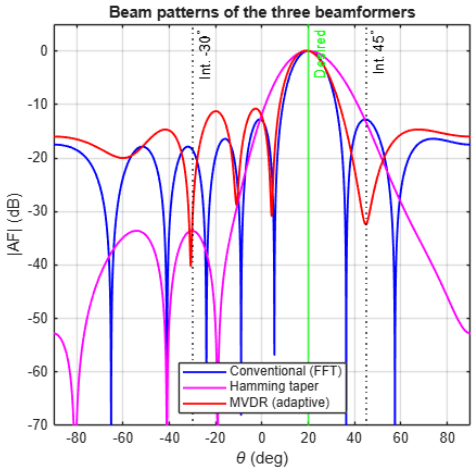
end
```

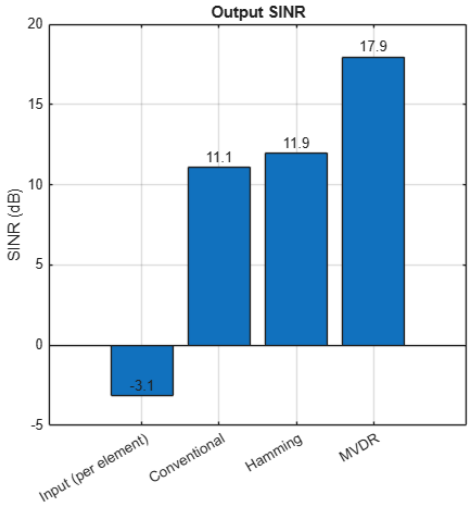



Beam pattern steered to 20° (peak at 20.0°)
|AF| (dB)
θ (deg)
Direct AF formula
FFT-based AF


Polar beam pattern (dB)


Beam steering with the same array (phase-shift weights)
|AF| (dB)
θ (deg)
Steer -30°
Steer 0°
Steer 20°
Steer 45°





---------- 7. OUTPUT PARAMETER TABLE ----------

```
fprintf('\n================ SETUP ================\n');

fprintf(['fc = %.0f GHz | lambda = %.3f mm | ' ...
    'd = lambda/2 = %.3f mm | N = %d\n'], ...
    fc/1e9, lambda*1e3, d*1e3, N);

fprintf(['Desired = %d deg | Interferers = %d and %d deg | ' ...
    'SNR = %d dB | INR = %d dB each\n'], ...
    thD, thI(1), thI(2), SNRdB, INRdB);

fprintf('\n=========== OUTPUT PARAMETER TABLE ===========\n');

fprintf('Spatial spectrum - detected peaks (deg) : %s\n', ...
    mat2str(round(doaFFT,1)));

fprintf('True directions (deg) : %s\n', ...
    mat2str(sort([thD thI])));

fprintf('Steering direction (max of |AF|) : %.2f deg (target %d deg)\n', ...
    mC.peak, thD);

fprintf('Steering direction from FFT pattern : %.2f deg\n', steerFFT);

fprintf('\n--- Beam pattern characteristics (steered to %d deg) ---\n', thD);

fprintf('%-22s %-10s %-18s %-12s\n', ...
    'Beamformer','HPBW (deg)','First nulls (deg)','PSLL (dB)');

fprintf('%-22s %-10.1f [%5.1f, %5.1f] %-12.1f\n', ...
    names{1}, mC.hpbw, mC.nulls(1), mC.nulls(2), mC.psl);
```

```
fprintf('%-22s %-10.1f [%5.1f, %5.1f] %-12.1f\n', ...
    names{2}, mH.hpbw, mH.nulls(1), mH.nulls(2), mH.psl);

fprintf('\n--- Steering cases: direction of maximum |AF| ---\n');

for i = 1:numel(steerCases)

    fprintf('Steering command %4d deg -> beam peak at %6.2f deg\n', ...
        steerCases(i), peakAng(i));

end

fprintf('\n--- Array gain (dB) at desired / interference directions ---\n');

fprintf('%-22s %-14s %-14s %-14s\n', ...
    'Beamformer','Desired 20','Interf. -30','Interf. 45');

for i = 1:3

    fprintf('%-22s %-14.1f %-14.1f %-14.1f\n', ...
        names{i}, gainAt(W{i}, thD), ...
        gainAt(W{i}, thI(1)), gainAt(W{i}, thI(2)));

end

fprintf('\n--- Output SINR ---\n');

fprintf('Input SINR (per element) : %.2f dB\n', SINR_in);

for i = 1:3

    fprintf('%-22s : %.2f dB (improvement %.2f dB)\n', ...
        names{i}, SINR_out(i), SINR_out(i)-SINR_in);

end
```

```
================ SETUP ================
fc = 28 GHz | lambda = 10.714 mm | d = lambda/2 = 5.357 mm | N = 8
Desired = 20 deg | Interferers = -30 and 45 deg | SNR = 10 dB | INR = 10 dB
each

=========== OUTPUT PARAMETER TABLE ===========
Spatial spectrum - detected peaks (deg) : [-30 20.2 44.8]
True directions (deg) : [-30 20 45]
Steering direction (max of |AF|) : 20.00 deg (target 20 deg)
Steering direction from FFT pattern : 19.99 deg

--- Beam pattern characteristics (steered to 20 deg) ---
Beamformer             HPBW (deg) First nulls (deg)  PSLL (dB)
Conventional (FFT)     13.6       [  5.3,  36.3] -12.8
Hamming taper          21.7       [-19.2,  90.0] -33.6

--- Steering cases: direction of maximum |AF| ---
```

```
Steering command  -30 deg -> beam peak at -30.00 deg
Steering command    0 deg -> beam peak at   0.00 deg
Steering command   20 deg -> beam peak at  20.00 deg
Steering command   45 deg -> beam peak at  45.00 deg

--- Array gain (dB) at desired / interference directions ---
Beamformer              Desired 20     Interf. -30    Interf. 45
Conventional (FFT)      -0.0           -18.6          -12.8
Hamming taper           0.0            -33.6          -13.4
MVDR (adaptive)         0.0            -30.6          -32.3

--- Output SINR ---
Input SINR (per element) : -3.14 dB
Conventional (FFT)      : 11.06 dB (improvement 14.20 dB)
Hamming taper           : 11.95 dB (improvement 15.09 dB)
MVDR (adaptive)         : 17.94 dB (improvement 21.08 dB)
```

---------- LOCAL FUNCTIONS ----------

```
function idx = top_peaks(y, num)

    y = y(:);

    loc = find(y(2:end-1) > y(1:end-2) & ...
        y(2:end-1) >= y(3:end)) + 1;

    [~, o] = sort(y(loc), 'descend');

    idx = loc(o(1:min(num, numel(loc))));

end

function m = beam_metrics(th, dB)

    dB = dB - max(dB);

    [~, ip] = max(dB);

    m.peak = th(ip);

    l = ip;

    while l > 1 && dB(l-1) >= -3
        l = l-1;
    end

    r = ip;

    while r < numel(th) && dB(r+1) >= -3
        r = r+1;
    end
```

```
    m.hpbw = th(r) - th(l);

    ln = ip;

    while ln > 1 && dB(ln-1) <= dB(ln)
        ln = ln-1;
    end

    rn = ip;

    while rn < numel(th) && dB(rn+1) <= dB(rn)
        rn = rn+1;
    end

    m.nulls = [th(ln) th(rn)];

    side = dB;

    side(ln:rn) = -inf;

    m.psl = max(side);

end

function s = sinr_db(w, Xd, Xi, Xn)

    Ps = mean(abs(w.'*Xd).^2);

    Pint = mean(abs(w.'*Xi).^2);

    Pn = mean(abs(w.'*Xn).^2);

    s = 10*log10(Ps/(Pint + Pn));

end
```

Published with MATLAB® R2026b